"Estructura del Proyecto - CLE"

Vega Ángeles Christopher

22140787

Moya Arreola Cristian Moya

22140757

Ruiz Avilés Fernanda Michelle

22140756

Casas Vázquez Luis Américo

22140750

Ingeniería en Sistemas Computacionales, Instituto Tecnológico de Querétaro.

Taller de Desarrollo Ágil de Software

José Miguel Jaime Ramírez

Índice

Organización de los Microservicios 3

Estructura de Directorios 3

Función de las Carpetas Principales y Archivos Raíz 4

Arquitectura de Capas Lógicas 4

Organización de los Microservicios

El ecosistema backend está fragmentado en cuatro microservicios independientes que gestionan dominios específicos del negocio:

usuarios-service

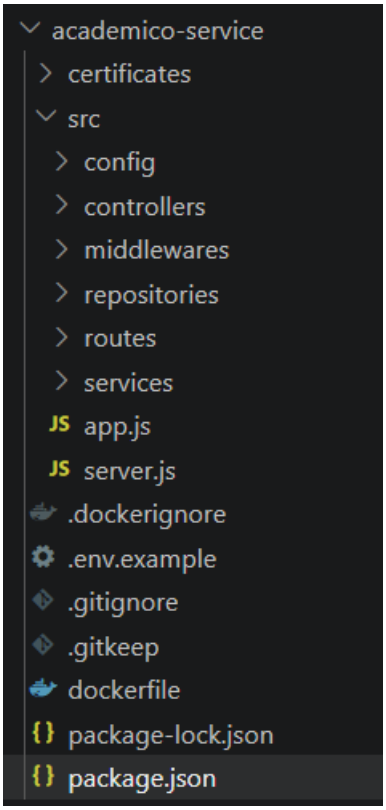
academico-service

infraestructura-service

inscripciones-service

Estructura de Directorios

Cada microservicio sigue un patrón de arquitectura en capas estandarizado. Tomando como referencia el módulo academico-service, el árbol de directorios es el siguiente:



Función de las Carpetas Principales y Archivos Raíz

certificates/: Almacena los certificados necesarios para establecer conexiones seguras (por ejemplo, certificados SSL/TLS para bases de datos o HTTPS interno).

src/config/: Contiene los archivos de configuración central, como las cadenas de conexión a PostgreSQL (Amazon RDS) y el manejo del entorno de despliegue.

src/middlewares/: Aloja funciones que interceptan las peticiones. Aquí se ubican los validadores de tokens JWT, filtros de control de acceso por roles (RBAC) y validadores de esquemas de datos.

dockerfile / package.json: Definen respectivamente cómo se construye el contenedor Docker del microservicio y las dependencias de Node.js necesarias para su ejecución.

src/app.js y src/server.js: Separan la configuración de la aplicación Express (app.js) del levantamiento del puerto y servidor HTTP (server.js), facilitando la ejecución de pruebas unitarias.

Arquitectura de Capas Lógicas

El directorio src/ implementa una arquitectura limpia basada en 4 capas fundamentales, cuyas funciones son:

src/routes/: Capa de Enrutamiento. Su función es definir las URLs (endpoints) expuestas por el microservicio y asignar los métodos HTTP (GET, POST, PUT, DELETE). Conecta las peticiones entrantes con sus respectivos middlewares de seguridad y validación antes de pasarlas al controlador. No contiene lógica de negocio.

src/controllers/: Capa de Control. Su función es actuar como puente entre las rutas y los servicios. Se encarga de recibir el *payload* (body, params, query), invocar la función correspondiente en la capa de servicios, y devolver la respuesta estructurada al cliente (frontend) junto con el código de estado HTTP (ej. 200, 400, 500).

src/services/: Capa de Negocio. Su función es centralizar el 100% de la lógica operativa del CLE. Aquí se ejecutan las reglas del negocio, se realizan validaciones de referencias cruzadas entre servicios, y se orquestan las llamadas a la base de datos a través de los repositorios.

src/repositories/: Capa de Acceso a Datos. Su función es aislar la lógica de negocio del motor de base de datos. Contiene exclusivamente las consultas SQL dirigidas a PostgreSQL. Recibe instrucciones de la capa de servicios para insertar, leer, actualizar o eliminar registros, devolviendo los datos crudos para que el servicio los procese.